

```
"""
```

```
Runs the three failure conditions from  
Section 7.1 of the SAW paper against  
both InterfaceLayerSystem and SEAGSystem,  
and reports MEASURED (not invented)  
rates of execution without a valid,  
persisted approval record.
```

```
Condition (a): automated test client  
bypassing the frontend confirmation.  
Condition (b): network interruption during  
approval presentation.  
Condition (c): simultaneous approval  
attempts by two users (race condition).  
"""
```

```
import threading  
import random  
import json  
from systems import fresh_db, ExecutionLog,  
InterfaceLayerSystem, SEAGSystem
```

```
N_TRIALS = 500 # per condition, per system
```

```
# -----  
-----  
# Condition A: automated bypass (skip the  
confirmation dialog entirely)  
# -----
```

```

-----
def run_condition_a():
    results = {}

    # -- Interface-layer baseline --
    log = ExecutionLog()
    sys_a = InterfaceLayerSystem(log)
    for _ in range(N_TRIALS):
        run_id = sys_a.create_run()

    sys_a.bypass_execute_directly(run_id) #
    attacker skips dialog entirely
    unauthorized = sum(1 for e in
    log.executions if not
    e["had_valid_approval"])
    results["interface_layer"] = {
        "trials": N_TRIALS,
        "executions": len(log.executions),
        "unauthorized_executions":
    unauthorized,
        "unauthorized_rate": unauthorized /
    N_TRIALS,
    }

    # -- SEAG --
    conn = fresh_db()
    log2 = ExecutionLog()
    sys_b = SEAGSystem(log2, conn)
    for _ in range(N_TRIALS):
        run_id = sys_b.create_run()

    sys_b.bypass_execute_directly(run_id) #
    attacker tries same bypass
    unauthorized2 = sum(1 for e in

```

```

log2.executions if not
e["had_valid_approval"])
    results["seag"] = {
        "trials": N_TRIALS,
        "executions": len(log2.executions),
        "unauthorized_executions":
unauthorized2,
        "unauthorized_rate": unauthorized2
/ N_TRIALS,
    }
    return results

# -----
-----
# Condition B: network interruption during
approval presentation
# -----
-----

def run_condition_b():
    results = {}
    random.seed(42)

    # -- Interface-layer baseline --
    log = ExecutionLog()
    sys_a = InterfaceLayerSystem(log)
    for _ in range(N_TRIALS):
        run_id = sys_a.create_run()

    sys_a.present_confirmation_dialog(run_id)
        network_ok = random.random() > 0.3
# 30% simulated network failure rate
        sys_a.user_clicks_approve(run_id,
backend_call_succeeds=network_ok)

```

```

        unauthorized = sum(1 for e in
log.executions if not
e["had_valid_approval"])
        results["interface_layer"] = {
            "trials": N_TRIALS,
            "executions": len(log.executions),
            "unauthorized_executions":
unauthorized,
            "unauthorized_rate": unauthorized /
N_TRIALS,
        }

# -- SEAG --
conn = fresh_db()
log2 = ExecutionLog()
sys_b = SEAGSystem(log2, conn)
random.seed(42)
executed_count = 0
for _ in range(N_TRIALS):
    run_id = sys_b.create_run()

sys_b.present_confirmation_dialog(run_id)
    network_ok = random.random() > 0.3
    ok = sys_b.approve_workflow(run_id,
network_fails=not network_ok)
    if ok:
        executed_count += 1
    unauthorized2 = sum(1 for e in
log2.executions if not
e["had_valid_approval"])
    results["seag"] = {
        "trials": N_TRIALS,
        "executions": len(log2.executions),
        "unauthorized_executions":

```

```

    unauthorized2,
        "unauthorized_rate": unauthorized2
    / N_TRIALS,
        "note": "executions blocked by
network failure are not unauthorized "
            "executions -- they are
correctly-blocked legitimate approvals "
            "(a availability cost, not
a safety violation). See discussion.",
        }
    return results

```

```

# -----
-----
# Condition C: simultaneous approval
attempts by two users (race condition)
# -----
-----

def run_condition_c():
    results = {}

    # -- Interface-layer baseline: no
locking, both threads can execute --
    log = ExecutionLog()
    sys_a = InterfaceLayerSystem(log)
    double_executions_a = 0
    for _ in range(N_TRIALS):
        run_id = sys_a.create_run()

    sys_a.present_confirmation_dialog(run_id)

    def click():

```

```

sys_a.user_clicks_approve(run_id,
backend_call_succeeds=True)

        t1 = threading.Thread(target=click)
        t2 = threading.Thread(target=click)
        t1.start(); t2.start()
        t1.join(); t2.join()

        run_executions = [e for e in
log.executions if e["run_id"] == run_id]
        if len(run_executions) > 1:
            double_executions_a += 1
        results["interface_layer"] = {
            "trials": N_TRIALS,
            "double_execution_incidents":
double_executions_a,
            "double_execution_rate":
double_executions_a / N_TRIALS,
        }

        # -- SEAG: DB transaction + status
precondition should prevent double exec --
        conn = fresh_db()
        log2 = ExecutionLog()
        sys_b = SEAGSystem(log2, conn)
        double_executions_b = 0
        for _ in range(N_TRIALS):
            run_id = sys_b.create_run()

sys_b.present_confirmation_dialog(run_id)

        def approve():
            sys_b.approve_workflow(run_id,
user_id=1)

```

```

        t1 =
threading.Thread(target=approve)
        t2 =
threading.Thread(target=approve)
        t1.start(); t2.start()
        t1.join(); t2.join()

        run_executions = [e for e in
log2.executions if e["run_id"] == run_id]
        if len(run_executions) > 1:
            double_executions_b += 1
        results["seag"] = {
            "trials": N_TRIALS,
            "double_execution_incidents":
double_executions_b,
            "double_execution_rate":
double_executions_b / N_TRIALS,
        }
        return results

if __name__ == "__main__":
    all_results = {
        "condition_a_automated_bypass":
run_condition_a(),
        "condition_b_network_interruption":
run_condition_b(),
        "condition_c_concurrent_approval":
run_condition_c(),
        "n_trials_per_condition": N_TRIALS,
    }
    print(json.dumps(all_results,
indent=2))

```

```
with open("results.json", "w") as f:  
    json.dump(all_results, f, indent=2)
```